

Who Needs Unit Tests Anyway?

Modernizing Legacy Code
With 0% Code Coverage

Jan Baart

2024



Jan Baart

Senior Software Engineer

- Energy Industry
- Desktop Applications
- Backend Services

SAP® Certified
Integration with SAP S/4HANA®

 **TÜVRheinland®**
Genau. Richtig.
ISO 27001 ISMS





What are legacy systems?

- Long-lived (> 10 years)
- Large (> 1 million LOC)
- Old-fashioned coding style
- Insufficient automated tests
- Poor documentation
- Outdated dependencies



What are they not automatically?

- Unsupported
- Obsolete
- Out of date
- In need of replacement
- Lacking modern features



Why this talk?

- Major pain point
- Costly
- Widespread
- Useful
- Valuable
- „Tested“



Why Modernize?



Negative Perspective

- Nightmare to read and change
- Life-threatening obsolete dependencies
- Nobody wants to work on it
- Timesink



Fears

- General fear of change
- Breaking existing code
- Alienating original developers
- High costs



Positive Spin

- Improve maintainability
 - Replace dependencies with modern alternatives
 - Empower new developers
 - Speed up development of new features
-
- Keep evolving the code and application
 - Increase stability and reduce brittleness
 - Collaboration between veterans and junior developers
 - Reduce maintenance costs and generate income



Human Factor



Psychological

- Start with why
- Show, don't tell
- Keep everyone in the loop
- Take veterans along
- Foster collaboration



Practical

- Create a dedicated team
- Find guru(s)
- Cultivate the right mindset
- Train new technologies
- Encapsulate cryptic/exotic code



Mindset

- It's not someone else's code, it's yours
- No blaming, we all write bad code
- Do you really want to write a replacement?
- Improving bad code makes you a better programmer
- Much better understanding of our language
- Might learn some clever tricks or even wisdom
- Working on a critical and extremely important project
- Your chance to become irreplaceable



Do not be afraid

- No unit tests does not mean no testing
- Breaking stuff also happens in fresh code
- Bugs are less likely to occur in deliberate refactorings than in forced changes to code you do not understand
- Modernized code is easier to fix
- Trust your skills!



How?



Spelunking* (aka making sense of it)

- Experience in reading complex code is required
- If at all possible, get a mentor
- Find your save havens
 - Areas which are relevant that you fully understand
- Use anchor points
 - Pieces of code you can set a breakpoint to detect what's going on
 - Logging, dialog base classes, helper function, db access etc.
- Learn to ignore fluff and find the core algorithm
- Refactoring can help, even if you discard it
- Use IDEs to your advantage (e.g. dependent breakpoints)

*credit for the term goes to @snemarch (Sune Marcher)



Declutter your codebase

- Remove any dead code or dependencies
- Get rid of leftover „TODO“ comments
 - Rename them or introduce new one if necessary
- Rename leftover nomenclature
 - Consistency in naming helps onboarding
 - Names should match the domain as it is currently defined
 - Gets rid of synonyms and conflicting names
 - Can be as simple as a global search & replace
- Remove useless macros
 - Can cause issues with tooling (e.g. IntelliSense)
 - Search & replace with regular expressions (or use tooling)



Fix existing code

- Enable /W4 (or equivalent) and actually handle all warnings
- Not as hard as you think
- Start with headers
- Use a branch and fix one diagnostic per commit only
- Allow disabling or suppressing specific warnings



Update the foundation

- Upgrade the compiler version and language version
 - Compiler version first
 - Version by version
 - Aim for the latest version
 - Handle all new warnings
- Update dependencies as far as possible



Homogenize your projects

- Use the same project settings as often as possible
 - Compiler version
 - Language standard
 - Warnings and errors
 - Optimization
 - Language extensions
 - Etc.



Set new standards

- Define a standard coding style
 - Prevents conflicts between original designers and juniors
 - Reduces mental burden when jumping within codebase
- Auto-format all code
 - New style will be very hard to use otherwise
 - Caveat: You lose (some) version control history



Introduce better tooling

- Better/newer IDE
- Static analysers
- Sanitizers as far as applicable
- Package management



Handle outdated APIs and types

- Create a layer for legacy API interaction
- Wrap outdated custom and API/framework types
 - Replace with modern standard type inside wrapper if possible
 - Otherwise extend for better use (e.g. iterators for custom containers)
- Deprecate usage of custom implementations
 - If they can be replaced by a library
 - Unless they exist for performance reasons
 - Additional features are still possible



Refactor on the fly

- Refactor small and cosmetic changes as you encounter them
- Gradually improves readability and stability
- Helps understand the code you want to change
- Enforcing separate commits for these hinders improvement



Refactor on the fly (Examples)

- Renaming variables or non-exported functions
- Marking local variables or single functions *const*
- Modern 1:1 replacements (e.g. *NULL* -> *nullptr*)
- Removing unnecessary antiquated idioms (e.g. *bool == 0*)

- Use guard clauses instead of nested *if*
- Replace simple classic *for* loops
- Replace single point of return logic with early returns
- Extract function



Refactor larger changes explicitly

- Refactor larger changes explicitly
- Pick your battles carefully
- Avoid refactoring just because you can
- One commit per refactoring and one refactoring per commit
- Focus on refactoring avoids unintended changes
- Separates refactoring and development
- Explicit change history
- Much easier to track down changed behavior



Document as you go

- Comment any existing code you had to analyse
- Comment missing domain context
- Comment larger refactorings
- Remove outdated comments
- Make sure you actually understand the code



Introduce unit testing

- Yes, 0% code coverage was a lie
- Try to introduce any unit testing you can
- Any framework is better than none
- The larger your refactoring, the more you need tests
- The smaller your refactoring, the easier it is to write a test

Book Recommendations

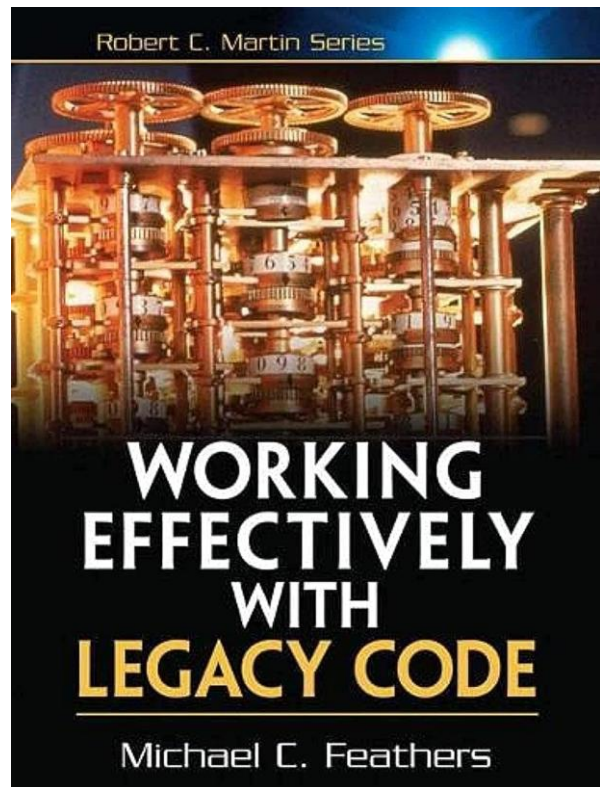
THE **LEGACY CODE** PROGRAMMER'S TOOLBOX

PRACTICAL SKILLS FOR DEVELOPERS
WORKING WITH LEGACY CODE



JONATHAN BOCCARA

FOREWORD BY KEVLIN HENNEY





THANK YOU!



www.bittner-krull.de



Bittner+Krull Softwaresysteme GmbH
Welfenstraße 31 A, D-81541 München



info@bittner-krull.de



Tel.: +49 89 458595-0



Need help with your legacy code?

<https://www.linkedin.com/in/janbaart>